DATA SCIENCE AND MACHINE LEARNING · END-SEMESTER PROJECT REPORT

ApexHunter 2.0

Finding Driver Mistakes and Pit-Stop Windows in Formula 1 Telemetry

Two seasons of real race data, two prediction problems, and every model tested honestly against a simple baseline

Submitted by

Parin Shah

23001091

Under the guidance of

[Guide name]

B Tech Computer Science

Navrachana University

November 2026

Abstract

This project asks two questions of Formula 1 telemetry, the data a car records several times a second: can a program find the moments where a driver made a **mistake**, and can it tell, lap by lap, **when a driver should change tyres**? We built **ApexHunter 2.0**, a complete system that downloads every race and qualifying session of the 2023 and 2024 seasons (92 sessions, 57.9 million telemetry rows) with the FastF1 library, cleans it, builds features and runs the models behind a Streamlit dashboard.

For **mistake detection** we compare a Z-score rule with an **Isolation Forest**. Every telemetry sample is described by how far it deviates from the five fastest other drivers at the same point of the track. Because real mistakes are not labelled, we plant known fake mistakes (lock-ups, early lifts, missed downshifts) in laps the model has not seen. The Isolation Forest finds more of them than the Z-score rule (mean F1 0.343 against 0.284) and is better at mistakes that combine several small deviations, such as lock-ups.

For **tyre strategy** the models predict how much slower the next lap will be. A **Ridge regression** baseline is compared with the deep-learning models of our second course. On five races held back for testing, Ridge is off by 0.344 s per lap against 0.357 s for simply repeating the last lap; the recurrent networks of the Deep Learning course do a little better still (0.332 ± 0.007 s for a vanilla RNN). A per-race pit-stop cost (median 22.8 s, 30 % cheaper under a safety car) turns these predictions into a five-lap pit window that we check against the stops drivers really made. The window contains the real stop about twice as often as a random window of the same size. A ceiling test shows that even perfect knowledge of tyre wear would not find more real stops: in modern Formula 1 the pit lap is set mainly by strategy and safety cars, not by tyre wear.

Keywords: anomaly detection, Isolation Forest, Ridge regression, Formula 1 telemetry, time series, model evaluation, FastF1

Contents

Abstract 2
Contents 3
1. Introduction 5
1.1 The problem 5
1.2 What we built 5
1.3 Main results 5
2. Key Ideas in Plain Words 6
2.1 Why this is hard 6
3. Data Collection 6
4. Data Cleaning 7
4.1 Cleaning the telemetry 8
4.2 Preventing leaks from the future 8
5. Processing the Data: Feature Engineering 8
5.1 Mistake features: compare with the field 8
5.2 Tyre features: what the last five laps say 8
5.3 Measuring the cost of a pit stop 9
5.4 A finding: race data cannot rank the compounds 9
6. The Models We Trained 10
6.1 Z-score baseline 10
6.2 Isolation Forest 10
6.3 Ridge regression 11
6.4 Recurrent networks (Deep Learning course) 11
6.5 From predictions to a pit window 11
6.6 What did not make it 11
7. How We Train and Test 12
7.1 Planted mistakes (mistake detection) 12
7.2 Held-out races (tyre models) 12
7.3 Rules that keep us honest 12
8. The Development Journey 13
9. Results 14
9.1 Mistake detection 14
9.2 Predicting the next lap 15
9.3 Pit windows on the test races 15
9.4 The ceiling test 16
10. How the System Works 17

11. Limitations and Future Work 17
12. Conclusion 17
References 18
Appendix A. Running the Project 18

1. Introduction

1.1 The problem

A Formula 1 car sends back speed, throttle, brake, gear, engine speed and position several times a second, and the FastF1 library makes this data public for every session since 2018. Teams employ engineers to read it; a fan or a junior engineer has no tool that points at the interesting moments. We framed two questions that can be answered from public data and, importantly, **tested**:

- **Mistakes.** In a driver's laps, where did he do something different from the fastest drivers at the same spot, in a way that cost time?
- **Pit stops.** From the laps driven so far, which laps form a good window for the next tyre change, and when a safety car comes out, should the driver stop now?

Both questions share the same difficulty: there is no answer key. No one labels mistakes, and the lap a team chose to stop is a strategic decision, not the "correct" answer. Most of the work in this project therefore went into building **honest tests**: planted mistakes at known places, races the models never saw, and simple baselines every model has to beat.

1.2 What we built

ApexHunter 2.0 is a pipeline from raw telemetry to a dashboard (Figure 1). This semester's work replaced last semester's mistake detector, which mostly flagged ordinary braking, and its tyre model, which had several data errors, with the methods described in this report. Each step is a set of Python scripts with automated tests that run on real data.

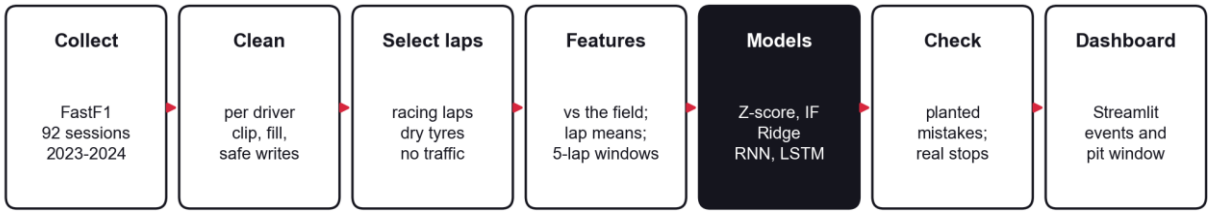


Figure 1. The ApexHunter 2.0 pipeline. Each box is a group of Python scripts in the project repository.

1.3 Main results

Table 1. Headline results. Mistake results are the mean of four sessions; tyre results are on five races (2024 rounds 19–24) that no model saw during training.

Question	Result	What it means
Mistakes: Isolation Forest vs Z-score	F1 0.343 vs 0.284	On planted mistakes, the machine-learning detector finds more (recall 0.27 vs 0.20) at similar precision (0.51 vs 0.55).
Next lap time: Ridge vs "no change"	0.344 s vs 0.357 s average error	Ridge is slightly better than repeating the last lap (4 % lower error, R^2 0.14). The RNN and LSTM (Deep Learning course) reach 0.328 and 0.335 s.
Pit window vs a random window	44 % vs 29 % of real stops	The five-lap window contains the real stop far more often than chance.
Ceiling test	Perfect tyre wear: 39 % vs 43 %	Even knowing the true tyre wear in advance does not find more real stops.

2. Key Ideas in Plain Words

No background in motor racing is needed for this report. These are the terms we use.

Table 2. Terms used in this report.

Term	Meaning in this report
Telemetry sample	One reading from the car: speed, throttle (0–100 %), brake (on/off), gear, engine speed (RPM) and position on track. About four per second.
Lap, stint	A lap is one trip round the track. A stint is the run of laps on one set of tyres, between two pit stops.
Pit stop, pit cost	A stop to change tyres. Its cost is the time lost against cars that did not stop: about 20–30 s depending on the track.
Safety car (SC), VSC	When the track is unsafe the field slows down behind a safety car, or to a set pace (virtual safety car). A stop then costs less, because everyone is slow.
Compound	The type of tyre: soft (fast, wears quickly), medium, hard (slow, lasts longer).
Representative lap	A lap at racing pace: not the first lap, not a pit or safety-car lap, not deleted by the stewards.
Anomaly	A data point that is unusual compared with the rest. A driving mistake is one kind.
Planted mistake	A fake mistake we insert at a known place in a real lap, so a detector can be scored on whether it finds it.
Precision, recall, F1	Precision: of the moments flagged, how many are real mistakes. Recall: of the real mistakes, how many were flagged. F1 combines the two (1 is perfect).
MAE, RMSE, R^2, MAPE	Error measures for numbers: mean absolute error, root mean squared error, share of variation explained, and error as a percentage of the lap time.
"No change" baseline	The simplest forecast: the next lap will take exactly as long as the last one. Any model must beat it to be useful.
Test races	The five most recent races, kept aside and used only once, to score finished models.

2.1 Why this is hard

- **There are no labels.** Nobody marks mistakes in telemetry, and a team's real pit lap is a decision made with information we do not have (radio, rivals, weather forecasts).
- **The signal is small.** Tyres lose about 0.05 s per lap of age, while lap times vary by about 0.9 s from one lap to the next because of traffic, wind and small errors.
- **Wear and fuel move together.** As a stint goes on the tyres get older and the car gets lighter at the same time, so their effects are hard to separate.

The aim was therefore never to build a perfect strategist, but to find what the data really supports and prove it is not luck, a data leak or overfitting.

3. Data Collection

All data comes from the **FastF1** Python library, which downloads Formula 1's official timing and car data. Table 3 lists what we collected.

Table 3. Data collected by the project.

Data	Source	Amount
Car telemetry: speed, RPM, gear, throttle, brake, X/Y/Z position, session time	FastF1 (download_season_data.py)	92 sessions (46 races, 46 qualifying), 2023–2024; 57.9 million rows, 1.8 GB
Lap data: lap and sector times, stint, compound, tyre age, pit in/out times, track status	FastF1 laps, read from the local cache	about 50,000 race laps
Gap to the car ahead and DRS state (for the traffic filter)	FastF1 car data	every race sample
Corner positions for event locations	FastF1 circuit information	every track

Telemetry is stored as one Parquet file per session in a local data lake. Last semester's Big Data work (Spark, HDFS and MongoDB copies of the same data) is kept but is not needed for this course; every model here reads the pandas output.

4. Data Cleaning

A model trained on wrong data gives wrong answers with complete confidence, so the first part of the semester was an audit of the whole code base. **47 problems** were logged and each was fixed with a test that fails on the old code. Table 4 lists those that would have changed the results of this report.

Table 4. Data problems that affected the models, and how each was fixed.

Problem we found	What it would have caused	How we fixed it
Mistake detector used absolute values (speed, brake)	95 % of "mistakes" were ordinary braking	Features measure the deviation from the fastest drivers at the same spot (Section 5.1)
Pit, safety-car and first laps mixed with racing laps	Slow laps flagged and learned as tyre wear	Only representative laps are used (a shared lap filter)
Intermediate and wet tyres mixed with dry tyres (2,460 laps)	Rain laps read as extreme wear	Dry tyres with a known tyre age only
Lap averages taken over irregularly spaced samples	About 1 km/h of noise in mean speed	Averages weighted by the time between samples
Slipstream and dirty air in race telemetry	Overtakes flagged as mistakes	Samples with DRS open or < 1 s behind another car are left out
Train/test split by random windows	Overlapping windows leaked into the test	Split by whole races, most recent races for testing
Scaler fitted on all data	Test statistics leaked into training	Scalers fitted on training races only
Mistake threshold tuned with a score that always picks the smallest value	Meaningless setting	Settings chosen with planted mistakes (Section 7)

4.1 Cleaning the telemetry

The cleaning script removes rows with no core values, fills short gaps within each driver, and clips physically impossible values (speed 0–380 km/h, RPM 0–15,000, throttle 0–100 %, brake 0–1). Files are written to a temporary name first and then renamed, so an interrupted run never leaves a half-written file. Telemetry samples arrive irregularly (0.014 to 0.76 s apart), which matters for every average taken later.

4.2 Preventing leaks from the future

- **Decisions use only the past.** A pit decision for lap k uses lap times up to lap k−1 and the track status of lap k; a prediction for lap k uses laps k−5 to k−1.
- **Whole races are held out.** Neighbouring five-lap windows share four laps, so a random split would put nearly identical windows on both sides.
- **The test races are used once**, to score finished models, never to choose settings.

5. Processing the Data: Feature Engineering

5.1 Mistake features: compare with the field

The key idea is to ask **"is the driver doing something unusual compared with what the fastest other drivers do at this exact point on track?"**, not "is this sample unusual?". Braking hard before Turn 1 is normal; braking hard on a straight is not. Every sample is placed on the track by projecting its position onto the session's fastest lap (a median 0.3 m away), and the five fastest other drivers give a profile per 10 m of track: median speed, throttle, gear and acceleration, their spread, and the share of them braking. Five features follow (Table 5), smoothed over 0.5 s.

Table 5. Mistake-detection features.

Feature	Formula	Catches
speed_deficit	(field speed − speed) / field spread	slow corner, lock-up, off-track moment
throttle_diff	(throttle − field throttle) / field spread	early lift, late throttle
brake_mismatch	brake on/off − share of the field braking here	braking where the field doesn't
gear_diff	(gear − field gear) / field spread	wrong gear, missed downshift
accel_diff	(acceleration − field) / field spread; over 0.5 s	sudden deceleration

Dividing by the field's spread matters: braking points vary by 10–20 m, so a raw speed difference is always large there. Without it, 73 % of flags were braking samples; with it, 39 %.

5.2 Tyre features: what the last five laps say

Each racing lap is reduced to nine numbers: lap time; time-weighted mean throttle, brake and RPM; tyre age; the compound (soft, medium, hard); and the race lap number. A model sees **five consecutive laps** and predicts the next one. Lap time and the three telemetry means are given relative to the last lap of the window, so the model sees how pace is changing rather than which track it is (absolute lap times mostly identify the track: copying the last lap already explains 99 % of their variation).

The target is the **change in lap time** from the last lap of the window to the next lap, in seconds (Figure 2). On 28,745 windows from 45 races it averages +0.02 s with a spread of 0.90 s; 98 % of changes are within ±2 s. Predicting "no change" is off by 0.39 s on average.

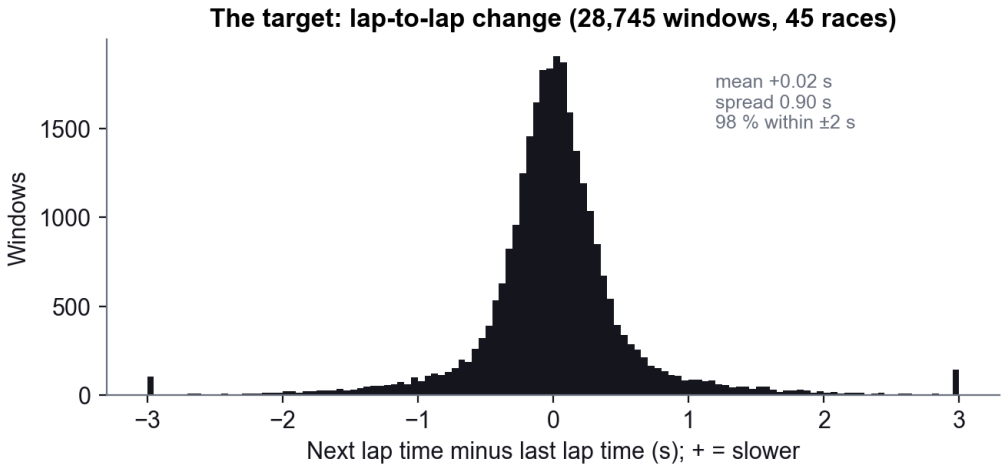


Figure 2. The tyre target: change in lap time from one lap to the next.

5.3 Measuring the cost of a pit stop

The cost of a stop is measured from each race's own laps: the driver's in-lap plus out-lap, minus the median of the same two laps for the cars that did not stop. Drive-through penalties (no tyre change) and first-lap damage stops are left out. Over 46 races the median normal stop costs **22.8 s** (from 18.5 s at Spa to 30.0 s in Qatar), and a stop under a safety car or VSC costs about **0.7×** as much (Figure 3). Each race uses the median of its own normal stops.

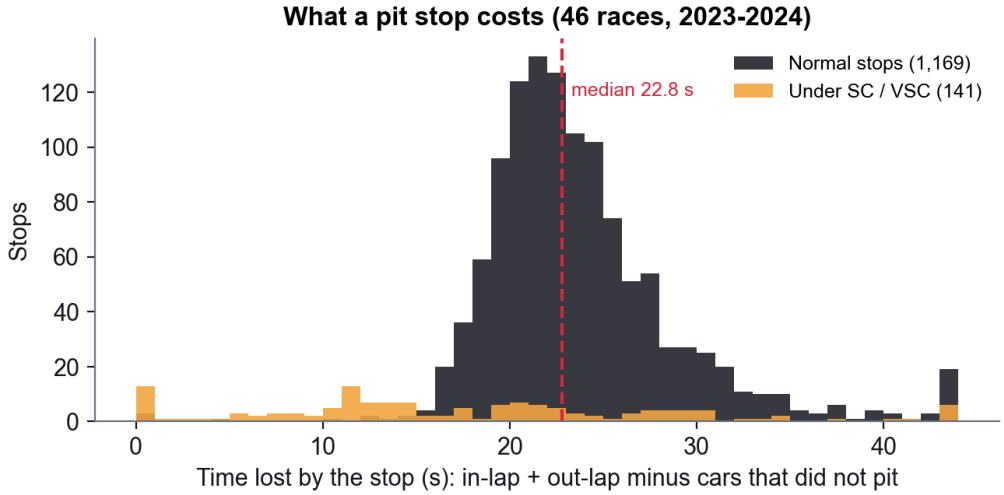


Figure 3. Time lost by a pit stop, normal and under a safety car or VSC.

5.4 A finding: race data cannot rank the compounds

We planned to let the strategy choose a tyre compound. A least-squares fit of lap time on a driver-race constant, compound, tyre age per compound and race lap showed that the compounds look almost the same in race data: the fresh-tyre gap between soft, medium and hard is 0–0.2 s, and its sign flips depending on which laps are included (Figure 4). Drivers save soft tyres, teams choose the compound that suits each stint, and "soft" means a different rubber (C1–C6) at every track. The strategy therefore uses the compound the driver actually fitted.

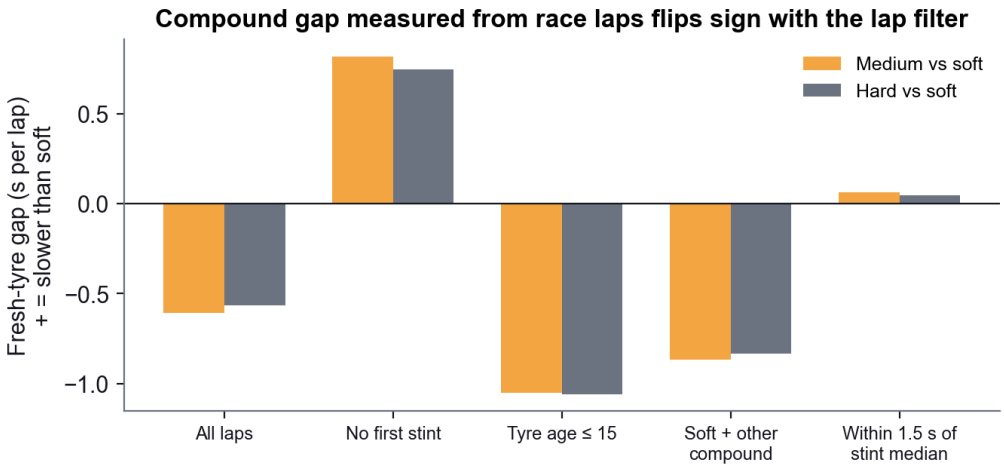


Figure 4. Fresh-tyre gap between compounds, fitted on 40 training races with different lap filters.

6. The Models We Trained

Table 6. Overview of the models. The recurrent networks are part of the Deep Learning course and are summarised here for completeness.

Model	Type	Problem	Role
Z-score rule	Statistics, no training	Mistakes	Baseline: is ML needed at all?
Isolation Forest	Unsupervised ensemble of random trees	Mistakes	Main DSML detector
"No change"	Rule, no training	Tyres	Baseline: next lap = last lap
Ridge regression	Regularised linear regression	Tyres	Main DSML model
Vanilla RNN, LSTM	Recurrent neural networks	Tyres	Deep Learning course comparison
Pit window	Calculation on the model's wear	Tyres	Turns predictions into advice

6.1 Z-score baseline

The Z-score rule learns each feature's mean and standard deviation on the training laps and scores a sample by its most extreme feature, $|z| = |x - \text{mean}| / \text{std}$. It needs no machine learning and shows how far a simple rule gets with the same features.

6.2 Isolation Forest

An Isolation Forest builds many random trees, each splitting on a random feature at a random threshold. Unusual points are separated from the rest after few splits, so the average path length to isolate a point is its anomaly score (Liu, Ting and Zhou, 2008). It needs no labels, which suits data with no answer key, and it can notice several moderate deviations at once.

- **Settings:** 200 trees, all rows per tree; max_features chosen from {1.0, 0.6} by F1 on planted mistakes.
- **Training data:** the driver's own racing laps (training on other drivers would make a slower driver look anomalous everywhere).

- **Threshold:** the 2 % most unusual samples are flagged (fixed for every session so counts compare across drivers).
- **Events:** flags less than 1 s apart are joined; runs of at least 0.5 s become events, each with a corner location, a reason in plain words and the time lost against the driver's own nearby laps.

6.3 Ridge regression

Ridge regression is linear regression with a penalty on large coefficients (Hoerl and Kennard, 1970):

$$\text{minimise} \quad \Sigma\,(y - X\beta)^2 \;+\; \alpha\,\Sigma\,\beta^2$$

Each five-lap window is flattened into 45 inputs and standardised inside the pipeline, so the scaler only ever sees training races. The penalty α was chosen by validation error on the most recent 20 % of the training races. A first grid stopped at 10,000 and picked its edge; widened, the best is **α = 30000**. Such a strong penalty is a sign that the inputs carry little stable signal: the model is mostly pulled towards predicting the average.

6.4 Recurrent networks (Deep Learning course)

A vanilla RNN and an LSTM (two layers, dropout 0.3, a linear output) read the same five-lap windows in order. They were trained with the same race split and metrics, three random starts each, so that the three models appear in one comparison (Section 9.2).

6.5 From predictions to a pit window

At every lap, every possible stop lap x of the stint is given a cost in seconds:

$$\text{cost}(x) = \text{wear} \times (\text{old-tyre ages on laps up to } x + \text{new-tyre ages after } x) + \text{pit cost}(x)$$

The sum runs to the end of the following stint (the driver's next real stop or the finish), so the driver's real number of stops is kept. Fuel, track and car are the same on a given lap whichever option is chosen, so they cancel; only tyre age and the lap of the stop differ. **Wear** per lap of tyre age is the average of the lap-time changes known so far in the stint plus 0.053 s for the fuel burnt each lap (fitted on the training races). The best stop lap is the one with the lowest cost, and the **window** is the best lap ±2 (five laps). On a safety-car lap the stop is 30 % cheaper, which often makes "pit now" the best option.

6.6 What did not make it

- **Choosing a compound.** Race data cannot separate the compounds (Section 5.4).
- **A window of all laps within 1 s of the best.** Because wear is small the cost curve is flat, and the window spanned 9 or more laps, sometimes half a stint.
- **Ten-lap windows for Ridge.** Lap-time error about the same as with five laps (0.343 vs 0.352 s for "no change"), but worse pit windows, because predictions start only at lap 12.
- **Wear from the last five laps only.** It jumped with lap-to-lap noise; the stint average replaced it.

7. How We Train and Test

7.1 Planted mistakes (mistake detection)

Laps are split into five folds by lap (GroupKFold). Each detector is fitted on the clean laps of four folds; in each held-out lap three fake mistakes of random types are planted (Table 8), with three random seeds. The detector's flags are turned into events with the same rule as the real output and scored per mistake.

Table 7. Fake mistakes used to score the detectors.

Fake mistake	Where	What is changed
Lock-up	inside a braking zone	an extra 15–30 km/h speed drop, then recovery
Early lift	just before the driver's own lift point	throttle off 0.5–1 s earlier, 3–8 km/h slower
Late throttle	after the driver re-applied throttle	throttle held at 0 for 0.5–1 s
Pedal overlap	in a braking zone	throttle 20–40 % while braking
Missed downshift	a corner below 150 km/h	one gear too high

A second check uses no fake data at all: for each event, the time lost over a 300 m stretch is compared with the same stretch on the driver's nearby laps, and the share of events that really cost time is compared with 200 random spots on the same laps.

7.2 Held-out races (tyre models)

The tyre data is split by whole races in date order: the 5 most recent races (2024 rounds 19–24, 3,006 windows) are the test set and the other 40 races train the models; settings are chosen on the latest 20 % of the training races. The pit-window check then runs each model's predictions through the window calculation on the test races and scores every source on the same stints.

Table 8. How results are measured.

Measure	How it is computed	Good result
Precision, recall, F1	Per planted mistake: is it inside an event; is an event on a mistake	Higher than the Z-score rule
Costly events	Share of events that lost time against the driver's own laps	Higher than random spots
MAE, RMSE	Error of the predicted lap time, in seconds	Lower than "no change"
R^2	Share of the lap-time change explained (on absolute lap times any model scores 0.99)	Above 0
MAPE	Error as a percentage of the lap time	Lower than "no change"
Window hit rate	Share of real stops inside the five-lap window	Above a random window of the same size

7.3 Rules that keep us honest

- **Every model faces a simple baseline** (Z-score; "no change"), and is reported even when it loses.

- **One test, used once.** Settings were never tuned on the test races.
- **Real-data tests.** 184 automated tests, none of which fake project code or data; integration tests run on real races from the cache, and each fix comes with a test that fails on the old code.
- **Random seeds.** Neural networks are trained three times and reported as mean ± spread.

8. The Development Journey

The final design came out of several iterations. We keep the dead ends because each taught us something used later.

Table 9. Iterations of the project.

#	What we did	What we found	Outcome
1	Audited last semester's code	47 bugs, from leaks to a detector that flagged braking	Fixed, each with a test
2	Mistakes: deviation from the field at the same spot	Braking share of flags 95 % → 37–54 %	Kept
3	Planted mistakes to choose settings	A score with an answer key replaced one that always picked the smallest value	Kept
4	Z-score baseline	Isolation Forest wins mainly on combined deviations (lock-ups)	Kept
5	Tyres: predict absolute speed	Copying the last lap scored R^2 0.994: it measured the track	Dead end
6	Predict the change in lap time	A target with a meaningful baseline (0.39 s)	Kept
7	Choose a compound	Race data cannot separate compounds	Dead end
8	Pit window within 1 s of the best	Windows of 9+ laps; two logic flaws found on real data	Replaced by ±2 laps
9	Ridge regression	Slightly better lap times, but no better pit windows than "no change"	Kept as baseline
10	Ceiling test with perfect wear	Perfect wear does not find more real stops	Key finding

The turning point (iteration 6). Predicting a lap's absolute speed looked excellent (R^2 0.99) only because every track has its own typical speed. Asking instead "how much slower will the next lap be?" removed the track and exposed how small the tyre signal really is, which is the honest starting point for everything after it.

9. Results

9.1 Mistake detection

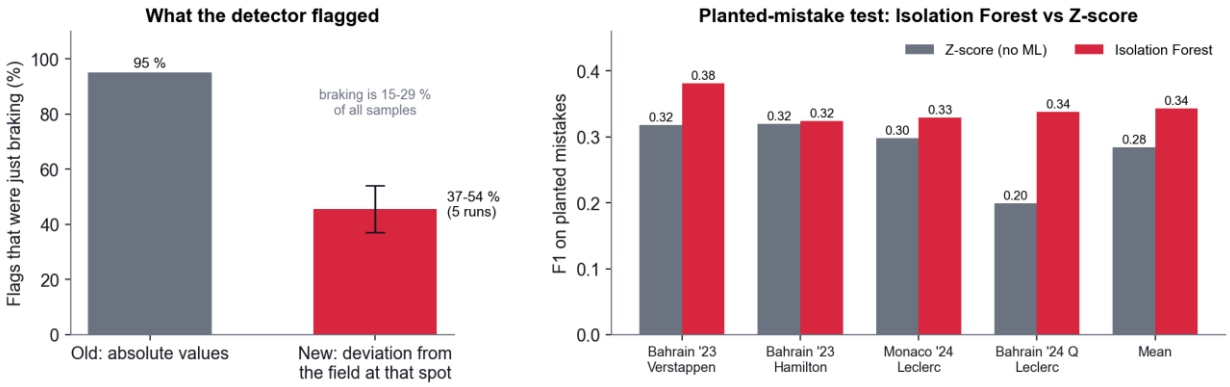


Figure 5. Left: share of flags that were just braking, before and after the rework. Right: F1 on planted mistakes, Z-score against Isolation Forest.

Table 10. Planted-mistake results per session (Z-score / Isolation Forest).

Session	Precision	Recall	F1	Costly events (random)
2023 Bahrain race, Verstappen	0.536 / 0.545	0.236 / **0.307**	0.318 / **0.381**	18 % / 15 % (10 %)
2023 Bahrain race, Hamilton	0.540 / 0.520	0.230 / 0.247	0.320 / 0.324	16 % / 19 % (12 %)
2024 Monaco race, Leclerc	**0.760** / 0.551	0.187 / **0.238**	0.298 / **0.329**	0 of 12 / 6 % (10 %)
2024 Bahrain qualifying, Leclerc	0.367 / 0.433	0.144 / **0.300**	0.199 / **0.338**	0 of 3 / 1 of 3 (19 %)
Mean	**0.551** / 0.512	0.199 / **0.273**	0.284 / **0.343**	

The Isolation Forest finds more planted mistakes in all four sessions. Its advantage comes from mistakes that combine several moderate deviations: it is better on lock-ups and missed downshifts in every session, while the Z-score rule is better on early lifts, where throttle alone is extreme (Figure 6). On real laps, in both Bahrain races the events cost time more often than random spots (15–19 % against 10–12 %); at Monaco they do not, because there many events differ from the field without costing Leclerc time against his own laps.

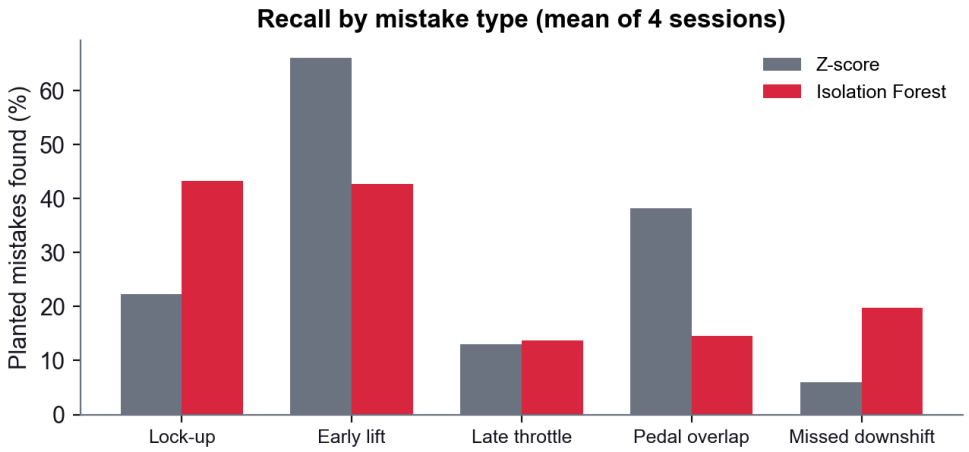


Figure 6. Share of planted mistakes found, by type (mean of four sessions).

9.2 Predicting the next lap

Table 11. Next-lap prediction on the 5 test races (3,006 windows). MAE and RMSE in seconds; R^2 of the lap-time change; MAPE of the lap time. The networks' MAE over three seeds: RNN 0.332 ± 0.007 s, LSTM 0.338 ± 0.004 s.

Model	MAE (s)	RMSE (s)	R^2	MAPE (%)	Train MAE (s)
"No change" (next = last lap)	0.357	0.593	0.00	0.392	–
Ridge regression (DSML)	0.344	0.549	0.142	0.377	0.388
Vanilla RNN (DL)	0.328	0.509	0.263	0.360	0.360
LSTM (DL)	0.335	0.517	0.238	0.368	0.363

Ridge beats "no change" by about 4 % on MAE and 7 % on RMSE. Its largest coefficients show what it learned: an unusually fast or slow last lap is followed by a move back to normal. Tyre age adds only about 0.001 s per lap, so it learned no wear that speeds up as the tyre ages. Training error is higher than test error because the test races were calmer, not because of overfitting.

The two recurrent networks beat Ridge on every measure (MAE over three seeds: RNN 0.332 ± 0.007 s, LSTM 0.338 ± 0.004 s; R^2 about 0.25 against 0.14). The plain RNN is level with the LSTM: with only five laps per window the gradients do not vanish, so the LSTM's gates bring no advantage. Both networks start to memorise after 10–17 epochs (training error keeps falling while validation error rises), so the weights of the best validation epoch are kept.

9.3 Pit windows on the test races

Table 12. Real stops inside the five-lap window, on the same 163 stints of the test races.

Wear from	Real stop in window	Random 5-lap window	Margin (points)	Real SC/VSC stops called
Laps driven (real lap times)	46.0 %	23.6 %	+22.4	100 %
"No change" (fuel only)	44.2 %	29.5 %	+14.7	77 %
Ridge	37.4 %	29.0 %	+8.4	67 %
Vanilla RNN	33.7 %	27.6 %	+6.1	62 %

LSTM	35.0 %	28.3 %	+6.7	67 %

Every source finds the real stop well above chance, but the trained models do not beat the simple sources: their lap-to-lap predictions correct noise rather than measure wear, and averaged over a stint they often predict laps getting faster, which gives no window at all. Figure 7 shows one stint: Russell in Singapore 2023, who stopped under the lap-20 safety car inside a window centred on lap 22.

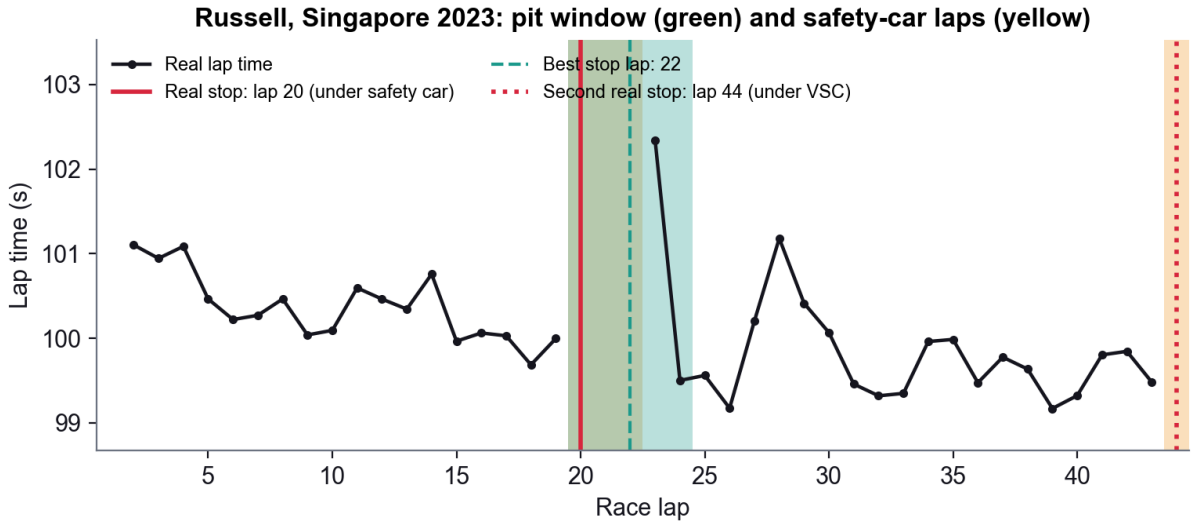


Figure 7. Pit window for Russell, Singapore 2023 ("no change" wear). Safety-car laps are yellow, the five-lap window green.

9.4 The ceiling test

To see how much a better tyre model could ever help, the window was given the **true** wear of every stint, measured from the whole stint including laps that had not happened yet. Over all 46 races (1,345 stints) it did no better than "no change" (Figure 8). Real stops are set by things outside a tyres-only model (undercuts, traffic, team plans and safety cars), and wear is so small that the best stop lap barely moves with it. Published work reaches a similar view: degradation rates are hard to separate in race data, and on F1 tyre data a tree model beat RNN, LSTM, GRU and transformer models (Todd et al., 2025).

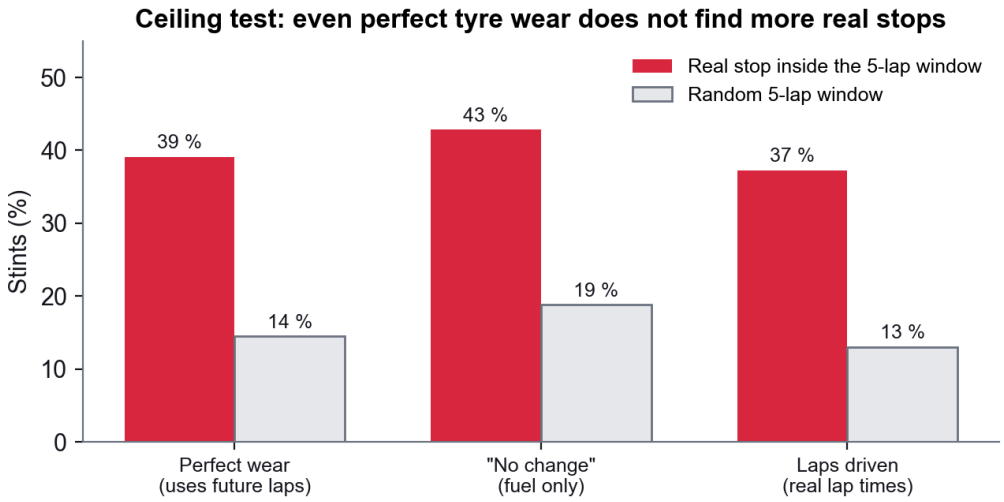


Figure 8. Ceiling test over 46 races: perfect wear against "no change" and the real lap times.

10. How the System Works

Each model is a command-line script that writes its output next to the data, and a Streamlit dashboard reads those files (Table 12).

Table 13. Main scripts and what they produce.

Script	What it does
download_season_data.py, clean_telemetry.py	Download and clean the 92 sessions
detect_mistakes.py --session … --driver … [--compare]	Isolation Forest events for one driver; --compare adds the Z-score baseline and the planted-mistake scores
train_ridge.py	Builds the tyre dataset, trains Ridge, scores it against "no change"
train_lstm.py --cell lstm | rnn	Trains the LSTM or the vanilla RNN, three seeds
check_strategy.py	Runs every model through the pit window on the test races; records the best source
predict_strategy.py --session … --driver …	Pit window, safety-car calls and real stops for one driver
streamlit run frontend/app.py	The dashboard: telemetry, AI analysis (mistake events and pit window), racing line, Big Data

On the dashboard's AI Analysis tab, the left card shows the mistake events on a track map with their corner, reason and time lost; the right card shows each stint's lap times with the pit window, safety-car laps, the best and the real stop, and a table comparing every wear source.

11. Limitations and Future Work

- **Fake mistakes.** The planted-mistake test measures "can it find these patterns"; it cannot prove that a flagged real moment was a mistake. The time-lost check covers part of that gap.
- **Few sessions scored.** Mistake detection was evaluated on four sessions, because each needs a full cross-validation run.
- **A tyres-only strategy.** Traffic, rivals and track position are ignored on purpose, which is why the ceiling is low: the models cannot see what really sets the pit lap.
- **Five test races.** Small differences between models (a few stints) are within noise.
- **Next steps.** An LSTM autoencoder for mistake detection (planned for the Deep Learning course, scored with the same planted-mistake test); predicting the laps until a driver pits directly; and outputs for every driver of every race.

12. Conclusion

We built a complete system that downloads two seasons of Formula 1 telemetry, repairs it, turns it into features and tests every model against a simple baseline on data it has not seen. For mistake detection, comparing each driver with the fastest drivers at the same point of the track turned a detector that flagged ordinary braking into one that finds planted mistakes, and the Isolation Forest finds more of them than a Z-score rule, mainly where several small deviations combine. For tyre strategy, Ridge regression predicts the next lap slightly better than repeating the last one, and a pit window built on a measured pit cost contains the real stop

about twice as often as chance. The most useful result is the honest one: even perfect knowledge of tyre wear would not find more real stops, because the pit lap is decided by strategy and safety cars.

Just as valuable were the habits the project taught: clean the data first, compare with the right reference, use tests with an answer key when the data has none, keep a simple baseline in every comparison, and report what does not work.

References

[1] F. T. Liu, K. M. Ting and Z.-H. Zhou, "Isolation Forest," in Proc. 8th IEEE Int. Conf. on Data Mining, 2008, pp. 413–422.

[2] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," Technometrics, vol. 12, no. 1, pp. 55–67, 1970.

[3] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[4] J. L. Elman, "Finding structure in time," Cognitive Science, vol. 14, no. 2, pp. 179–211, 1990.

[5] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[6] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems 32, 2019.

[7] P. Schaefer et al., FastF1: a Python package for accessing and analyzing Formula 1 results, schedules, timing and telemetry, github.com/theOehrly/Fast-F1, 2023.

[8] J. Todd et al., "Explainable time series prediction of tyre energy in Formula One race strategy," arXiv:2501.04067, 2025.

[9] "A state-space approach to modeling tire degradation in Formula 1 racing," arXiv:2512.00640, 2025.

[10] M. Kuhn and K. Johnson, Applied Predictive Modeling. New York: Springer, 2013.

Appendix A. Running the Project

On Windows with Python 3.13, from the project folder:

```
python -m venv .venv && .venv\Scripts\pip install -r requirements.txt
python backend/scripts/etl/download_season_data.py
python backend/scripts/etl/clean_telemetry.py
python backend/scripts/detect_mistakes.py --session data_lake/clean_data/2023_1_R.parquet --driver 1 --compare
python backend/scripts/train_ridge.py
python backend/scripts/train_lstm.py --cell lstm --force   (and --cell rnn)
python backend/scripts/check_strategy.py
python backend/scripts/predict_strategy.py --session data_lake/clean_data/2024_22_R.parquet --driver 1
python backend/scripts/tests/run_tests.py --all
streamlit run frontend/app.py
```